Make training work

VLC = Vision Learning Control

Practical Training of Deep Networks

Normalisation, stabilisation, and diagnosis

Jonathon Hare

Vision, Learning and Control
University of Southampton

Topics

- Initialisation, normalisation, and residual pathways
- Gradient clipping, learning-rate schedules, and weight decay
- Reading training curves and inspecting internal signals
- A systematic workflow for diagnosing failed training runs

Training is a coupled dynamical system

For parameters $\boldsymbol{\theta}_t$, an optimiser produces an update

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t + \Delta\boldsymbol{\theta}_t, \qquad \Delta\boldsymbol{\theta}_t = \text{update}(\nabla_{\boldsymbol{\theta}}\mathcal{L}_t, \text{optimiser state}).$$

- The gradient depends on the data, architecture, current parameters, loss, and numerical precision.
- The effect of the gradient depends on the optimiser, learning rate, and parameter scale.
- A symptom such as a flat loss curve therefore has several possible causes.

The practical question

What evidence would distinguish the possible causes?

Sanity checks before a long training run

- Inspect a minibatch: shapes, dtypes, value ranges, targets, and class frequencies.
- Check the contract between the model output and the loss. For example, `CrossEntropyLoss` expects raw logits and integer class indices.
- Verify that every intended parameter receives a finite gradient.
- Disable augmentation and regularisation, then overfit one small minibatch.
  - Failure usually points to the implementation, optimisation, or data pipeline rather than generalisation.
- Compare against a simple baseline before investing in a large run.

Initialisation recap

- Random weights break symmetry between units.
- Their scale should match the activation and the layer's fan-in and fan-out.

Activation	Typical choice	Weight variance
Linear or tanh	Xavier / Glorot	$2/(n_{\text{in}} + n_{\text{out}})$
ReLU	He / Kaiming	$2/n_{\text{in}}$

- Sensible initialisation keeps early activations and gradients at a useful scale.
- It only controls the start of training. Parameter updates can still move the network into a poorly scaled regime.
- PyTorch's historical `nn.Linear` default is not the standard He initialisation for ReLU.

Normalisation controls activation scale

Suppose a layer produces pre-activations $\boldsymbol{z}$. A normalisation layer constructs

$$\hat{\boldsymbol{z}} = \frac{\boldsymbol{z} - \boldsymbol{\mu}}{\sqrt{\boldsymbol{\sigma}^2 + \epsilon}}, \qquad \boldsymbol{y} = \boldsymbol{\gamma} \odot \hat{\boldsymbol{z}} + \boldsymbol{\beta}.$$

- The choice of axes used to compute $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}^2$ defines the normalisation method.
- The learned scale $\boldsymbol{\gamma}$ and shift $\boldsymbol{\beta}$ allow the layer to recover a useful representation.
- Normalisation changes the optimisation problem. It does not merely standardise the final representation.
- The small constant ϵ prevents division by zero and affects numerical behaviour when the variance is tiny.

Batch normalisation

For feature or channel k and a minibatch containing m values:

$$\mu_k = \frac{1}{m}\sum_{i=1}^{m} z_{ik}, \qquad \sigma_k^2 = \frac{1}{m}\sum_{i=1}^{m}(z_{ik} - \mu_k)^2,$$

$$y_{ik} = \gamma_k \frac{z_{ik} - \mu_k}{\sqrt{\sigma_k^2 + \epsilon}} + \beta_k.$$

- For a fully connected layer, statistics are normally computed over the batch for each feature.
- For a convolutional layer, they are normally computed over the batch and spatial positions for each channel.
- Each example therefore influences the normalised representation of the other examples in its minibatch.

BatchNorm during training and evaluation

Training mode

- Normalises using the current minibatch statistics.
- Updates running estimates of the mean and variance.
- Output for one example depends on the other examples in the minibatch.

Evaluation mode

- Normalises using stored running statistics.
- Gives a stable result for one example independently of the evaluation batch.
- Does not update the stored statistics.

PyTorch state matters

`model.train()` and `model.eval()` change BatchNorm behaviour. They do not enable or disable gradient recording.

Batch size and data dependence

- Small or unrepresentative minibatches give noisy estimates of the activation statistics.
- Large changes in batch size can therefore change the optimisation behaviour even when the learning rate is unchanged.
- Correlated examples within a batch can bias its statistics.
  - Examples include neighbouring video frames or patches from the same image.
- Running statistics can become inaccurate when fine-tuning on a small dataset or when the deployment distribution changes.
- Freezing parameters does not automatically freeze BatchNorm statistics. The module's training or evaluation mode controls them.

Layer normalisation

For one example with d features, LayerNorm computes

$$\mu = \frac{1}{d}\sum_{j=1}^{d} z_j, \qquad \sigma^2 = \frac{1}{d}\sum_{j=1}^{d}(z_j - \mu)^2.$$

- Each example supplies its own statistics, so the result does not depend on the other examples in the minibatch.
- The same computation is used during training and evaluation. There are no running statistics.
- LayerNorm is therefore convenient for variable-length sequences and small batches.
- The normalised axes still matter. In PyTorch they are specified by `normalized_shape`.

A useful distinction

BatchNorm normalises each feature or channel across examples. LayerNorm normalises features within each example.

Residual pathways preserve a direct route

A residual block computes

$$\boldsymbol{y} = \boldsymbol{x} + F(\boldsymbol{x}; \boldsymbol{\theta}), \qquad \frac{\partial \boldsymbol{y}}{\partial \boldsymbol{x}} = \boldsymbol{I} + \frac{\partial F}{\partial \boldsymbol{x}}.$$

- The identity term provides a direct route for activations and gradients.
- The residual branch learns a correction to the current representation rather than a complete replacement.
- Training is often more stable when the residual branch starts with a relatively small scale.
  - One practical method initialises the final normalisation scale in the branch to zero, so the block initially behaves approximately like the identity.
- Initialisation, normalisation, and residual structure therefore address related but distinct sources of poor signal propagation.

Gradient clipping limits unstable updates

For global norm clipping with threshold τ:

$$\tilde{\boldsymbol{g}} = \begin{cases} \boldsymbol{g}, & \|\boldsymbol{g}\|_2 \leq \tau, \\ \tau \dfrac{\boldsymbol{g}}{\|\boldsymbol{g}\|_2}, & \|\boldsymbol{g}\|_2 > \tau. \end{cases}$$

- Norm clipping preserves the gradient direction while limiting its magnitude.
- Value clipping clamps individual coordinates and can change the direction substantially.
- Apply clipping after `backward()` and before `step()`:

```
loss.backward()
torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
optimizer.step()
```

- Frequent clipping is evidence to investigate, not proof that the underlying instability has been fixed.

Learning-rate schedules and warm-up

- The learning rate controls the scale of the parameter update, rather than the size of the gradient itself.
- Warm-up starts with a small learning rate and increases it over the first part of training.
  - This limits early updates while activations, gradients, and optimiser state are still settling.
- Decay then reduces the learning rate as training approaches a useful solution.
  - Step decay makes discrete reductions.
  - Cosine decay changes the learning rate smoothly towards a chosen minimum.
- Record the learning rate alongside the loss. A scheduler called at the wrong frequency can otherwise be difficult to diagnose.

Ordering in PyTorch

For the usual per-step schedule: `optimizer.step()` before `scheduler.step()`.

Weight decay and optimisation

Decoupled weight decay applies a direct shrinkage to the parameters:

$$\boldsymbol{\theta}_{t+1} = (1 - \eta_t \lambda)\boldsymbol{\theta}_t - \eta_t \text{ update}(\boldsymbol{g}_t).$$

- For plain SGD, adding an L2 penalty to the loss gives an equivalent update under the usual assumptions.
- Adaptive optimisers rescale gradient coordinates, so an L2 gradient penalty and direct weight decay need not have the same effect.
- AdamW makes the decay explicit and separate from the adaptive gradient update.
- Biases and normalisation scale and shift parameters are often excluded from decay, depending on the training recipe.
- Weight decay can help, but it does not guarantee good generalisation.

Training curves are diagnostic evidence

- **Training loss does not fall:** first check the data, loss contract, gradients, learning rate, and ability to overfit one minibatch.
- **Training loss falls but validation performance worsens:** investigate the split, distribution shift, augmentation, and regularisation.
- **Loss suddenly spikes or becomes non-finite:** inspect the offending minibatch, learning rate, activation scale, gradient norms, and numerical precision.
- **Training changes at an unexpected epoch:** compare the event with scheduler steps, data-pipeline changes, checkpoint restoration, and mode changes.

Keep the raw evidence

Log training and validation losses, task metrics, learning rate, gradient norms, and the exact configuration used for the run.

Inspecting activations, gradients, and updates

Useful summaries for each layer include:

- activation mean, standard deviation, range, and fraction of exact zeros;
- gradient norm and the fraction of non-finite values;
- parameter norm and the relative update

$$\frac{\|\Delta\boldsymbol{\theta}\|_2}{\|\boldsymbol{\theta}\|_2 + \epsilon};$$

- how these quantities change over training, rather than only their value at one step.
- A ReLU unit is dead when its pre-activation remains non-positive, so its output and local gradient remain zero.
- Sigmoid and tanh units saturate at large $|z|$, where their derivatives become small.
- Forward and backward hooks can collect these summaries without changing the model computation.

A systematic failure-diagnosis workflow

1. Reproduce the failure with a fixed configuration and retained logs.
2. Inspect the data and the model–loss contract.
3. Reduce to the smallest case that still fails.
4. Overfit a small batch and inspect activations, gradients, and parameter updates.
5. Form a specific hypothesis, then change one factor at a time.
6. Restore complexity gradually from a known working baseline.

The goal

Replace blind hyperparameter search with evidence about where the training process first becomes unhealthy.